

# PostgreSQL hinter den Kulissen

Von Transaktionen  
bis Clustering

DigitalXChange, 17. September 2025



# Urban Spelling Bee

## Word List



## User Input

Delete

Your word goes in here

Try

## Your Score

0

## Instructions

- 🐛 Create as many words as you can by clicking on the given letters.
- 🐛 The word must be at least 4 letters long.
- 🐛 The word must contain the letter in the middle.
- 🐛 All letters can be used multiple times.
- 🐛 If the word is found in the dictionary, you're provided with its definition and the word is added to your word list.



Ariadne Engelbrecht



Oliver Tigges

# Grundlagen & Dauerhaftigkeit

| Subscriptions |         |         |                   |           |
|---------------|---------|---------|-------------------|-----------|
| ID            | User_ID | Card_ID | Product           | Term      |
| 1             | 31      | 47      | Streaming Plus    | yearly    |
| 2             | 27      | 60      | Streaming Plus    | quarterly |
| 3             | 22      | 51      | Fitness App       | monthly   |
| 4             | 28      | 54      | Cloud Storage Pro | yearly    |
| 5             | 20      | 42      | Fitness App       | monthly   |
| 6             | 19      | 37      | Fitness App       | yearly    |
| 7             | 39      | 22      | Fitness App       | yearly    |
| 8             | 49      | 22      | Streaming Plus    | quarterly |
| 9             | 45      | 12      | Music Unlimited   | monthly   |
| 10            | 39      | 22      | Meal Kit Weekly   | quarterly |

| Credit cards |          |                     |             |
|--------------|----------|---------------------|-------------|
| ID           | Owner_ID | Number              | Valid_until |
| 1            | 2        | 4386 0886 9552 2    | 05/2029     |
| 2            | 4        | 4175 1598 0581 0426 | 02/2027     |
| 3            | 4        | 3565 6614 4625 1167 | 10/2028     |
| 4            | 7        | 3822 5786 7288 07   | 08/2026     |
| 5            | 1        | 2131 8604 9020 829  | 05/2029     |
| 6            | 1        | 1800 3496 5177 802  | 10/2027     |
| 7            | 1        | 4160 5288 1445 0    | 02/2026     |
| 8            | 4        | 3414 4311 0712 155  | 11/2025     |
| 9            | 7        | 2718 7270 7139 4156 | 09/2026     |

| Users |               |                                   |                        |
|-------|---------------|-----------------------------------|------------------------|
| ID    | Name          | E_Mail                            | Mobile_phone           |
| 1     | Hannah Adams  | hayesdustin@hotmail.com           | 001-760-430-2442       |
| 2     | Marc Nichols  | angeladuncan@mullins-allison.info | 001-243-903-5045x92202 |
| 3     | Joseph Martin | christiandavidson@alvarez.net     | 001-695-806-8056x526   |
| 4     | Robin Valdez  | pduncan@gmail.com                 | 829.724.9995x81349     |
| 5     | Sarah Gross   | amypayne@garcia-rogers.com        | 421-675-7543x34927     |
| 6     | Susan Brown   | colebruce@hotmail.com             | 6163784763             |
| 7     | Mark Jordan   | nrivas@valdez-holland.com         | 368.389.2658x562       |

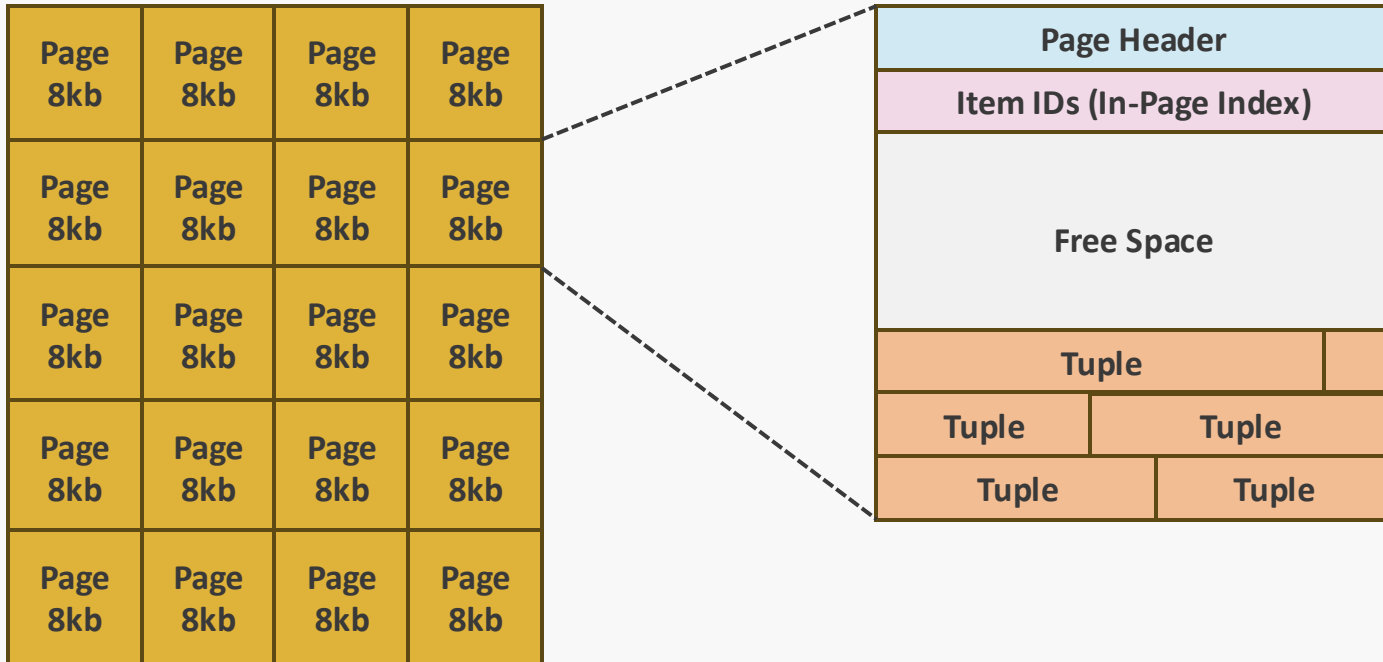
```

SELECT *
FROM subscriptions s
JOIN users u ON s.User_ID = u.ID
LEFT JOIN credit_cards c ON s.Card_ID = c.ID;

```

```
SELECT pg_relation_filepath('users');
```

Datei: /var/lib/postgresql/data/base/12621/16321



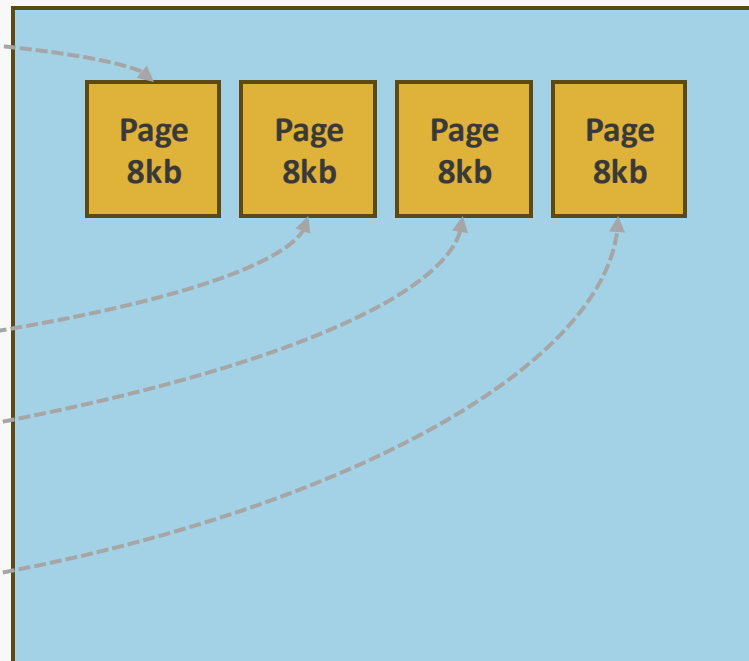
## Disk

### Database file

|             |             |             |             |
|-------------|-------------|-------------|-------------|
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |

## RAM

### Shared Buffer Cache



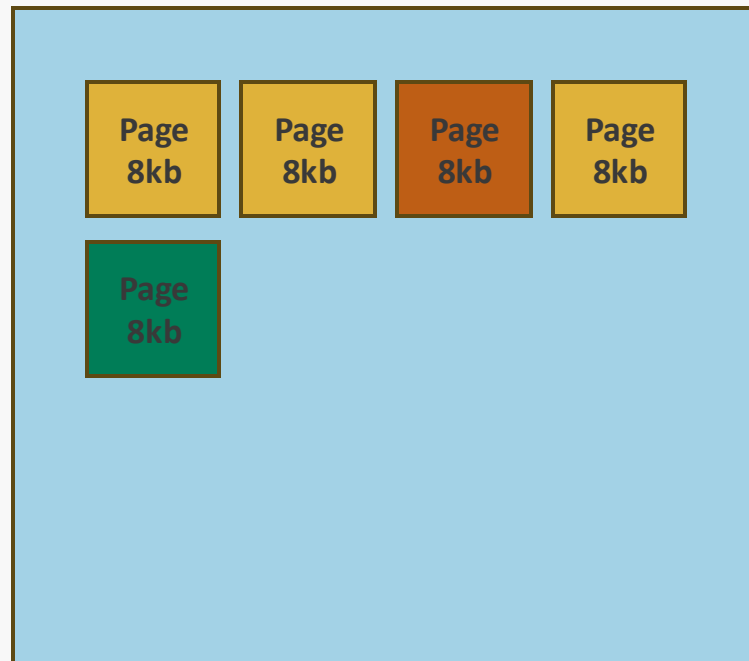
## Disk

### Database file

|             |             |             |             |
|-------------|-------------|-------------|-------------|
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |

## RAM

### Shared Buffer Cache



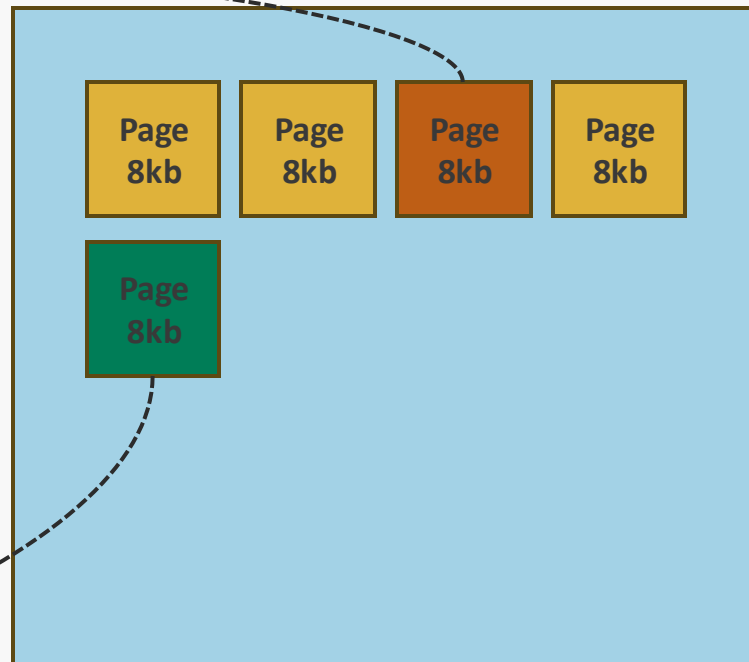
## Disk

### Database file



## RAM

### Shared Buffer Cache



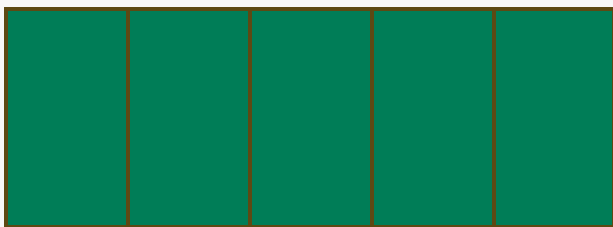
# Was passiert wenn der Server crasht?

## Disk

### Database file

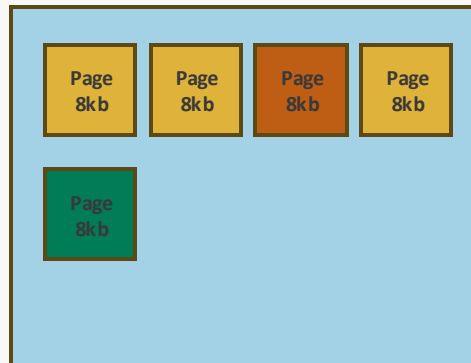
|             |             |             |             |
|-------------|-------------|-------------|-------------|
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |

### Write Ahead Log (WAL)



## RAM

### Shared Buffer Cache



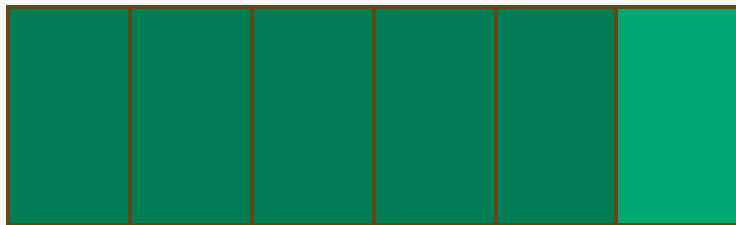
LSN: 0/165B1298  
TXID: 52489  
Operation: INSERT  
Table: public.subscriptions  
Tuple:  
  ID = 71  
  User\_ID = 12  
  Card\_ID = 37  
  Product = 'Streaming Plus'  
  Term = 'monthly'  
  Page: 345 (offset 3)

## Disk

### Database file

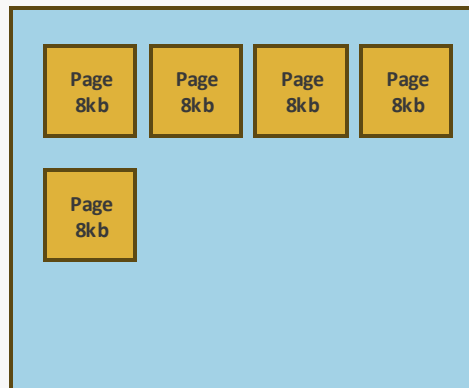
|             |             |             |             |
|-------------|-------------|-------------|-------------|
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |
| Page<br>8kb | Page<br>8kb | Page<br>8kb | Page<br>8kb |

### Write Ahead Log (WAL)



## RAM

### Shared Buffer Cache



# Effizienz

Der Postbote überlegt sich  
den besten Weg.

# Wir stellen uns DHL vor

| Postbote                                                                                                |                  | Was passiert                                                  | PostgreSQL                    |
|---------------------------------------------------------------------------------------------------------|------------------|---------------------------------------------------------------|-------------------------------|
|  Suche ohne Navi       | Ohne Index       | Postbote fährt die ganze Stadt ab, liest jedes Straßenschild  | Seq Scan<br>(Full Table Scan) |
|  Routenplan erstellen  | CREATE INDEX     | Postbote trägt in sein Navi die Position aller Straßen ein    | Index erstellen               |
|  Effiziente Zustellung | Mit Index suchen | Postbote schaut ins Navi, springt direkt zur gesuchten Straße | Index Scan                    |

# Live Demo

# Top 3 Indices

| Index-Typ     | Typischer Einsatz                                     |
|---------------|-------------------------------------------------------|
| <i>B-Tree</i> | Standard für Gleichheit und Range ( $=$ , $<$ , $>$ ) |
| <i>GIN</i>    | Volltextsuche, JSON, Arrays                           |
| <i>GiST</i>   | Geodaten, komplexe Strukturen                         |

Don't index **blindly**.



**Schnelleres Lesen:**

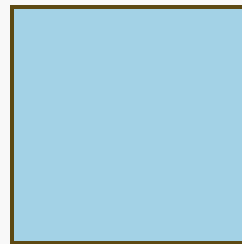
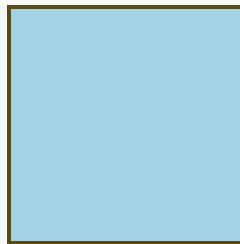
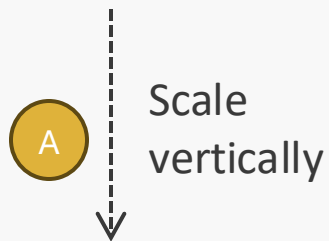
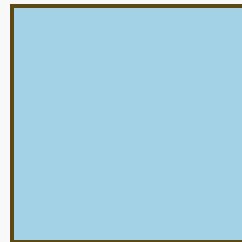
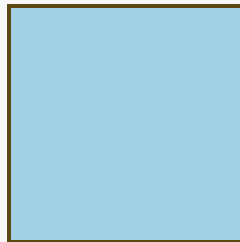
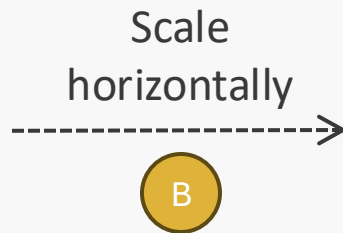
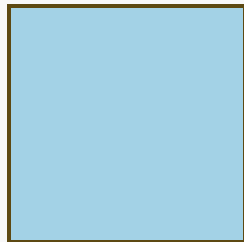
- Suchen (SELECT)
- Sortieren (ORDER BY)



**Langsameres Schreiben  
(INSERT, UPDATE, DELETE)**

# Replikation & Skalierung

PostgreSQL  
Server



# 1. Partitionen

## Server 1

Partition 1:  
Bestellungen 2023

Partition 2:  
Bestellungen 2024

Partition 3:  
Bestellungen 2025

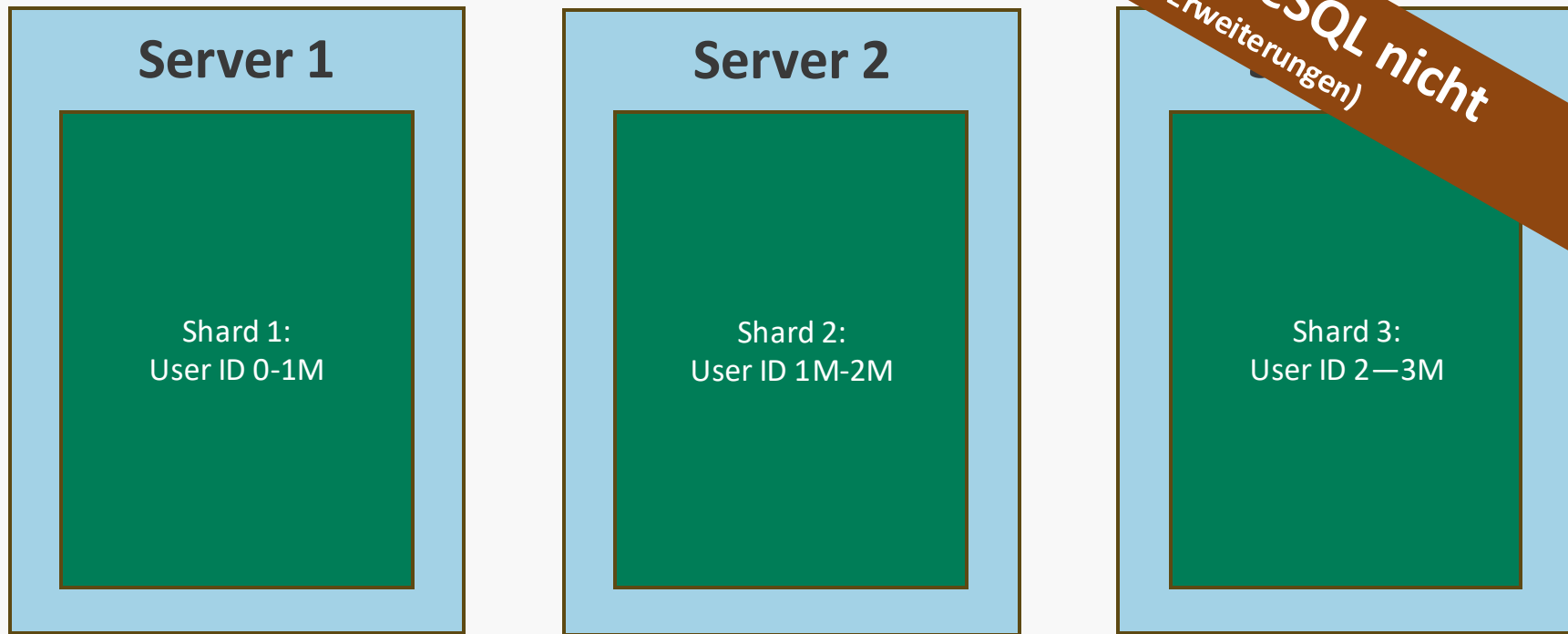
```
CREATE TABLE orders (  
  order_id    bigserial PRIMARY KEY,  
  customer_id bigint NOT NULL,  
  order_date  date NOT NULL,  
  total_amount numeric(10,2)  
) PARTITION BY RANGE (order_date);
```

```
CREATE TABLE orders_2023 PARTITION OF orders  
  FOR VALUES FROM ('2023-01-01')  
  TO ('2024-01-01');
```

```
CREATE TABLE orders_2024 PARTITION OF orders  
  FOR VALUES FROM ('2024-01-01')  
  TO ('2025-01-01');
```

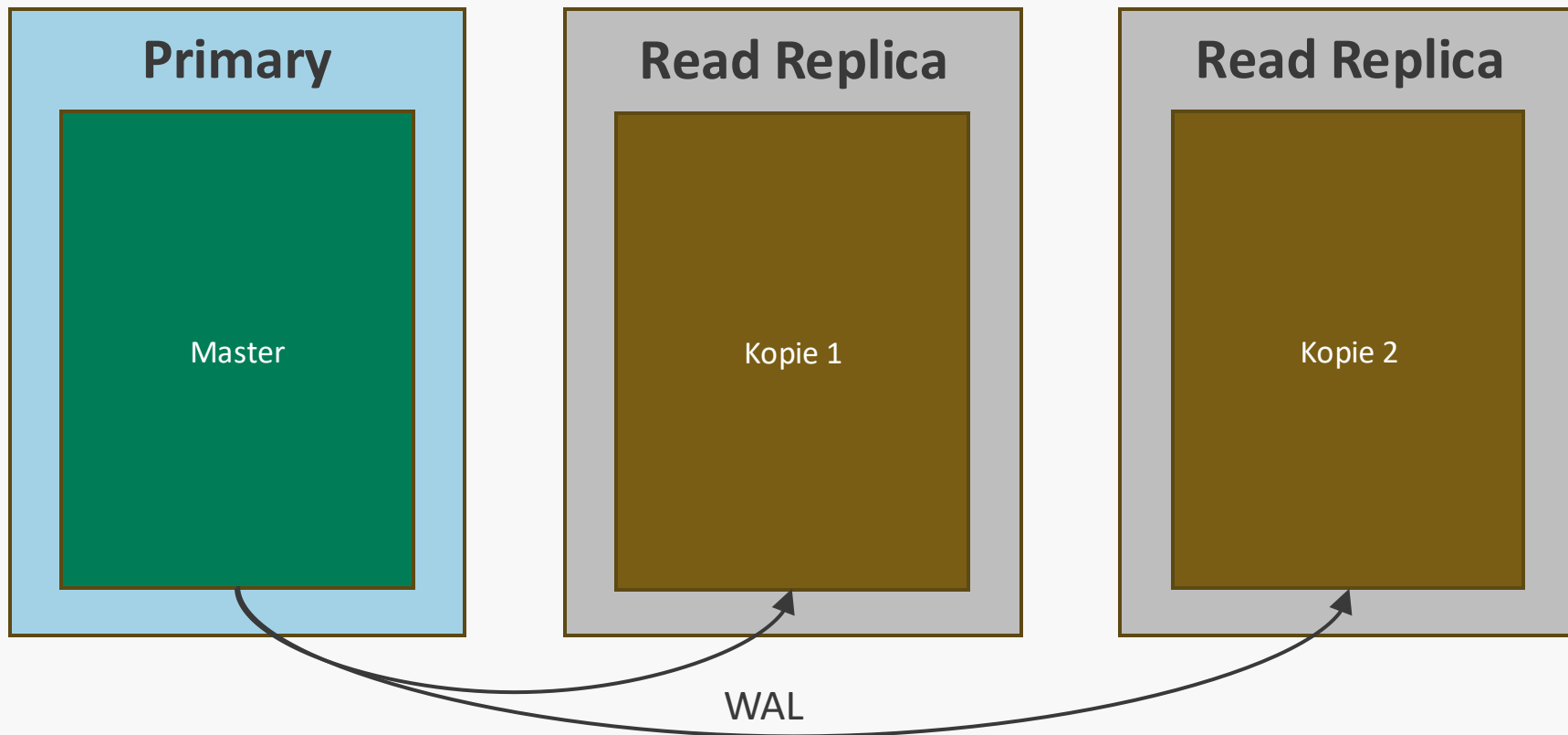
```
CREATE TABLE orders_2025 PARTITION OF orders  
  FOR VALUES FROM ('2025-01-01')  
  TO ('2026-01-01');
```

## 2. Sharding



### 3. Replikation

Performance vs. Konsistenz



```
import psycopg2
```

```
# Verbindung zu Primary
```

```
write_conn = psycopg2.connect(  
    host="primary-db.example.com",  
    ...  
)
```

```
def update_user_email(user_id, new_email):
```

```
    with write_conn.cursor() as cur:
```

```
        cur.execute(  
            "UPDATE users SET email = %s WHERE id = %s",  
            (new_email, user_id)  
        )  
        write_conn.commit()
```

```
# Verbindung zu Read Replica
```

```
read_conn = psycopg2.connect(  
    host="read-replica-db.example.com",  
    ...  
)
```

```
def get_user_by_id(user_id):
```

```
    with read_conn.cursor() as cur:
```

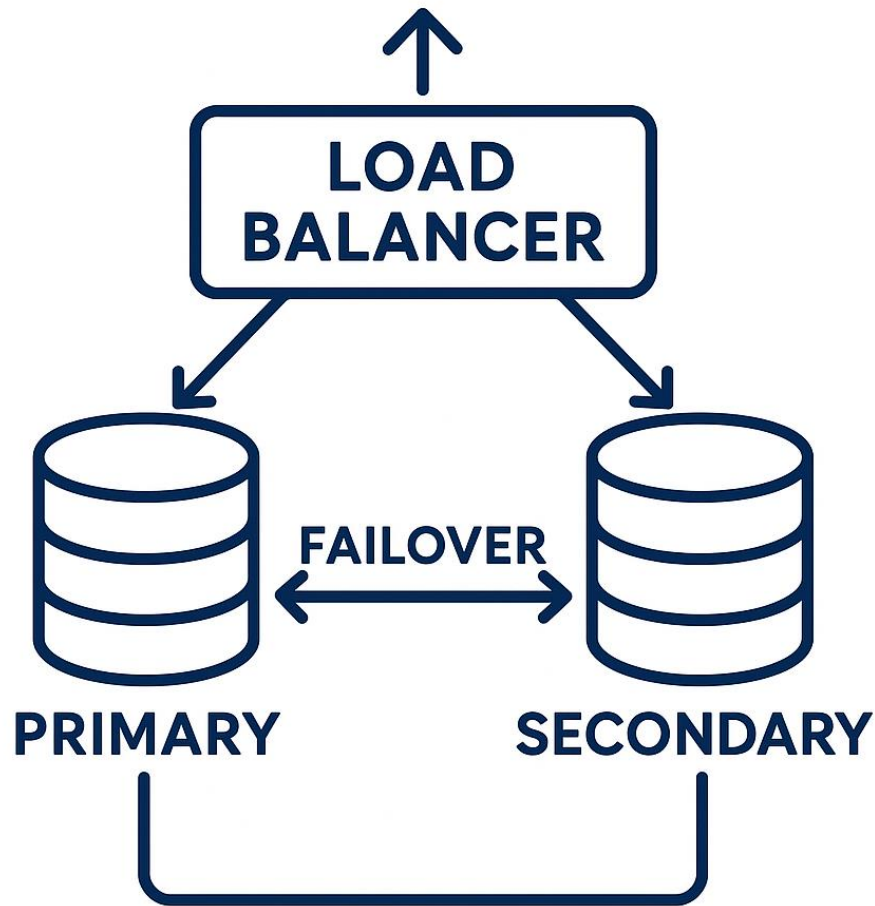
```
        cur.execute(  
            "SELECT id, name, email FROM users WHERE id = %s",  
            (user_id,))  
        return cur.fetchone()
```

## 4. Cluster

Cluster besteht aus **mehreren DB-Instanzen**

Agieren gemeinsam  
als **eine logische Datenbank**

**Automatisches Failover**



# Takeaway Skalierung

Horizontale vs. Vertikale Skalierung

Begriffe: Partition, Shard, Replika, Cluster

Replikation selbst kontrollieren oder Cluster-Service nutzen

Trade-Off: Performance vs. Konsistenz

# Konsistenz

**Erst an der Kasse wird's  
offiziell.**

# Wir sind im Supermarkt

| Supermarkt-Aktion                                                                                                                | PostgreSQL             | Was passiert                     |
|----------------------------------------------------------------------------------------------------------------------------------|------------------------|----------------------------------|
|  Einkaufswagen nehmen                           | BEGIN;                 | Transaktion startet              |
|  Produkte in den Wagen legen oder zurückstellen | INSERT, UPDATE, DELETE | Änderungen sammeln sich          |
|  An der Kasse bezahlen                          | COMMIT;                | Alle Änderungen werden dauerhaft |
|  Einkauf abbrechen                              | ROLLBACK;              | Alles wird rückgängig gemacht    |

# Anna und der letzte Käse

```
1  -- Warenkorb nehmen
2  BEGIN;
3
4  -- Anna legt Käse, Brot und Wein in den Warenkorb
5  INSERT INTO einkauf (kunde, artikel, menge) VALUES ('Anna', 'Käse', 1);
6  INSERT INTO einkauf (kunde, artikel, menge) VALUES ('Anna', 'Brot', 1);
7  INSERT INTO einkauf (kunde, artikel, menge) VALUES ('Anna', 'Wein', 1);
8
9  -- Anna überlegt es sich nochmal → entfernt das Brot
10 DELETE FROM einkauf WHERE kunde = 'Anna' AND artikel = 'Brot';
11
12
13 -- Lager mit Constraint (Bestand darf nicht negativ werden)
14 UPDATE lager SET bestand = bestand - 1 WHERE artikel = 'Käse';
15 UPDATE lager SET bestand = bestand - 1 WHERE artikel = 'Wein';
16
17 -- Checkout, erfolgreiches Bezahlen an der Kasse
18 COMMIT;
19
```

# ACID

## **Atomicity**

Entweder der ganze Einkauf oder gar nichts.

## **Consistency**

Bestände stimmen nach dem Bezahlen.

## **Isolation**

Regelt, dass nur einer den letzten Käse kauft.

## **Durability**

Bezahlte Waren gehören Dir für immer.

# Takeaways



PostgreSQL wächst mit euren Anforderungen



# Danke

Welche Fragen habt Ihr?